Subject-Enforced Completeness for AI Inspection Logs

Atlas Project
tomas.asin.gonzalez@gmail.com

Working paper — June 19, 2026
arXiv cs.CR (preprint)

Abstract

Key transparency systems such as CONIKS [9] and the IETF `keytrans` working-group drafts [4] allow users to monitor their own entries in an append-only log and detect equivocation by the operator — without relying on global monitors. We transfer this pattern to a new domain: **completeness of AI content-inspection logs**.

For a given subject's request stream, we show that completeness of the inspection log — whether every inspectable request was in fact inspected — is **unilaterally falsifiable** by the subject, without waiting for an external auditor. The mechanism binds each inspection to a signed, monotonically-sequenced request emitted from the subject's device. A gap in the subject's own sequence is a proof of omission: the operator cannot skip an inspection and suppress the evidence unilaterally.

We implement the mechanism on RFC 9162 [7] (Certificate Transparency v2), extend it symmetrically to output inspection (every model response committed before delivery, verified client-side), and provide a reference implementation. Layers 3–4 extend the scheme to external witnesses, split-view detection, and behavioral drift observation. We state ten honest limits, including split-view exposure, geolocation irresolvability, and covert post-inspection capability restrictions.

We do not claim a new cryptographic primitive. The contribution is *domain transfer*: carrying the subject-monitoring model of CONIKS and Key Transparency from key-binding logs to AI content-inspection logs, and modelling *inspection-by-cause* as the invariant to enforce. Unlike prior work on agent auditability — which assumes honest infrastructure operators — our mechanism is designed specifically for the case where the operator controls both the AI pipeline and the inspection log. The reference implementation comprises 1831 tests and is mypy `--strict` clean.

Contents

1. Introduction: Integrity ≠ Completeness

Existing transparency infrastructure for AI systems focuses on *integrity*: proving that log entries have not been altered after the fact. Merkle trees and signed tree heads (STHs) provide this guarantee efficiently, and the Certificate Transparency ecosystem has deployed it at scale.

Integrity, however, is the wrong property for inspection logs. An operator who wants to hide that a request was processed without inspection does not alter existing log entries; they simply *omit* the new one. A Merkle tree that grows correctly is consistent with an operator who never inserts certain inspections. Verifying that the log has not been tampered with says nothing about whether it is complete.

The gap matters structurally. In AI deployments, the entity that operates the model is also the entity that operates the inspection infrastructure. This is not a malicious arrangement — it is the natural vertical integration of a model provider — but it creates a conflict of interest that integrity proofs do not address: the same party who decides whether to run an inspection is the party who controls what the inspection log records.

Our starting observation is that completeness, unlike integrity, has a per-subject structure. A user who sends one hundred requests has a ground truth: one hundred signed requests left their device. If the operator's log acknowledges only ninety-eight inspections, two are unaccounted for. The user does not need a regulator, a second operator, or a global monitor to compute this. They need their own counter.

The regulatory context sharpens the motivation. EU AI Act Articles 12, 13, and 53 impose record-keeping and transparency obligations on AI providers. When a regulator asks "was this request inspected?", the current answer is "trust the operator's log." We ask whether the answer can be "the subject's device can verify it independently."

Fable 5 / next-generation frontier models (Claude, GPT, Gemini) operate in a regulatory grey zone: powerful enough to fall under systemic-risk obligations, deployed widely enough that individual inspection gaps aggregate into policy-relevant failures. This is deployment context motivating urgency, not a technical claim about any specific model. We mention it in this section only.

2. Threat Model

2.1. The Operator Is Automated Infrastructure

A critical clarification: **the operator in this model is the AI provider's automated infrastructure** (API gateway, inspection pipeline, log service), not an individual human employee making per-request decisions. In a typical AI API deployment, every request from every user flows through the same automated pipeline without manual intervention.

This distinction matters for the threat model. Omissions arise from:

- **Configuration exceptions**: certain traffic patterns or API paths are routed around the inspection module.
- **Silent pipeline degradation**: the inspection service crashes or is rolled back; the gateway continues serving requests with a fallback that omits inspection.
- **Deliberate policy gaps**: the operator configures the system not to inspect certain categories of request by definition.

We model the operator as **semi-honest**: the infrastructure is operated by a company that wants to demonstrate compliance (regulatory pressure, contractual obligations), and the log

infrastructure runs honestly for the requests it handles. The adversarial capability is that the company controls which requests enter the inspection pipeline at all.

The adversary's two goals:

1. **Selective omission**: process a request without creating an inspection record for it.
2. **Split-view**: show the subject a log view that includes certain inspection records while showing a different view to the regulator.

2.2. What the Adversary Controls

The operator controls: the inspection pipeline configuration; the log infrastructure; and the client distribution channel.

2.3. What the Adversary Does Not Control

- **The subject's device-bound key.** A key generated inside the device's TPM or Secure Enclave (cf. Apple App Attest, Android Key Attestation) is hardware-bound and non-exportable.
- **The subject's local counter state.** The monotonic counter is maintained by the client on the subject's device.
- **The subject's local log replica.** The twin replica (§3.4) is stored on the subject's device.

2.4. What We Do Not Model

We explicitly do not address: (a) an operator who is actively malicious and fully non-compliant; (b) physical compromise of the subject's device; (c) a subject whose "independent" client was actually distributed by the operator (§6, limit 6.3); (d) content inspected out-of-band after retention without a new signed request (limit 6.2).

3. Mechanism: Subject-Enforced Completeness

3.1. Device-Bound Request Signing

At first use, the subject authenticates via a federated identity provider (OAuth 2.0 / OIDC). At registration time, the client silently generates an asymmetric key in the device's hardware security module (TPM on PC/server, Secure Enclave on mobile). The private key never leaves the hardware boundary.

Thereafter, every request the subject sends carries a `CosignedRequest`:

```
CosignedRequest = {
    seq:          int,    # monotonic counter, starts at 0, never
        repeats
    payload_hash: str,    # SHA-256(prompt_bytes), hex
    signature:    str,    # device_key.sign(
        canonical_JSON({payload_hash, seq}) )
}
```

The canonical form signed is the compact, key-sorted JSON of `{payload_hash, seq}` — no timestamps (timestamps are provided by the operator's log entry, not by the subject, to avoid clock-skew disputes).

Why not WebAuthn/Passkey per-request. WebAuthn passkeys sign a server-provided challenge in response to a user gesture; they cannot sign arbitrary payloads silently. The correct

primitive for silent per-request signing is a device-bound key in the platform's HSM (Apple App Attest for mobile; client certificates bound to platform TPMs for desktop).

3.2. Inspection Binding and the In-Path Timing Guarantee

For every `CosignedRequest` that triggers an inspection, the operator records:

```
InspectionRecord = {
    seq:          int,    # from CosignedRequest
    payload_hash: str,    # SHA-256(payload) -- matches
        CosignedRequest.payload_hash
    cosig:        str,    # full CosignedRequest, JSON-serialised
    decision:     str,    # "allow" | "block" | "inspect"
    cause:        str,    # policy rule or heuristic that triggered
        this event
    timestamp_ns: int,    # operator-assigned epoch in nanoseconds
    salted_hash:  str,    # SHA-256(salt_i || payload) -- GDPR Art. 17
        erasure hook
}
```

Dual-hash leaf for GDPR Art. 17 (OSM-007). The `InspectionRecord` carries two hashes. `payload_hash` is the hash the subject already signed; it enables binding check 4 and is permanent. `salted_hash` is derived from a 32-byte random salt s_i stored *outside* the Merkle tree in a deletable store (`SaltStore`). To exercise the right of erasure, the operator destroys s_i; the committed salted hash becomes a dead-end without affecting tree integrity, consistency proofs, or `detect_omission()`.

The critical invariant: *an inspection without an antecedent signed request is invalid; a signed request without a corresponding inspection record is a detectable omission.*

In-path timing guarantee. Because the filter is always-on and mandatory in the API path, the `InspectionRecord` is committed to the Merkle log *before* the request is forwarded to the AI model. The model response cannot arrive at the client before the commitment exists.

The API response carries proof back to the client:

```
APIResponse = {
    result:             bytes,          # model response (opaque)
    seq_ack:            int,            # sequence the operator
        acknowledged
    leaf_bytes:         bytes,          # the committed
        InspectionRecord
    leaf_index:         int,
    inclusion_proof:    list[bytes],    # RFC 9162 inclusion proof
    sth:                SignedTreeHead, # {tree_size, root_hash, sig}
    consistency_proof: list[bytes],     # append-only proof from last
        head
    consistency_from:   int,
}
```

On each response the client runs six independent checks:

1. **STH signature** — the head is authentically the operator's.
2. **Append-only** — the log was not rewritten since the client's last head.
3. **Input inclusion** — the committed `InspectionRecord` is in the tree.
4. **Input binding** — `payload_hash` matches what the client signed.
5. **Output inclusion** — the committed `OutputInspectionRecord` is in the same tree.

6. **Output binding** — `SHA-256(result)` matches `output_hash` in the committed output record.

3.3. `detect_omission()`

The subject maintains a local set of every `seq` value they have emitted:

```
def detect_omission(observed_seqs: Sequence[int], last_emitted: int)
    -> list[int]:
     """Return seq values the operator did not acknowledge in [0,
         last_emitted]."""
     observed_set = set(observed_seqs)
     return [s for s in range(last_emitted + 1) if s not in
         observed_set]
```

This call requires no network request, no cooperation from the operator, and no cryptographic computation beyond what the subject already holds locally. A non-empty return value is a *unilateral proof of omission.*

3.4. Twin Independent Log Replicas

The operator maintains the authoritative Merkle log (RFC 9162). The subject maintains a lightweight local replica: an ordered list of `(seq, operator_root_at_seq)` tuples, updated with each acknowledged inspection. Consistency proofs verify append-only growth at each step.

What this does not close. The subject verifies their own view of the log. They cannot prove that the regulator sees the same STH. Full closure requires external witnesses (RFC 9162 §5 gossip). The HTTP witness transport is implemented (`HttpWitnessTransport` in `transparency/gossip.py`, Ed25519-verified counter-signatures, `has_quorum()`); deployment of independent witness nodes remains infrastructure work outside this paper's scope. The residual split-view exposure is stated as honest limit 6.

4. Implementation

The reference implementation lives in `src/atlas/transparency/` and is intentionally decoupled from any specific AI product.

`merkle_tree.py` — RFC 9162 §2.1 faithful. Domain-separation prefixes prevent second-preimage attacks:

```
leaf_hash(data)         = SHA-256(0x00 || data)
node_hash(left, right) = SHA-256(0x01 || left || right)
```

Exports `merkle_root()`, `inclusion_proof()` / `verify_inclusion()`, `consistency_proof()` / `verify_consistency()`. All operations are pure functions; the module has no I/O or state.

`log.py` — `TransparencyLog` wraps the Merkle primitives with append-only semantics, maintains the current STH, and signs it. Persistence is opt-in: `TransparencyLog(path=Path(...))` writes each leaf as a base64-encoded line with `fsync()` on append and reloads from disk on startup, preserving monotonic sequence continuity across process restarts. A Read-API for deployers and regulators exposes the log over HTTP: `GET /api/exec/api/v1/log/tree` (current STH), `GET /api/exec/api/v1/log/entries` (leaf range, capped at 1000), and `GET /api/exec/api/v1/log/proof/inclusion/{i}` (RFC 9162 inclusion proof), providing the technical basis for EU AI Act Art. 26 deployer monitoring.

`client_cosign.py` — `ClientCosigner` emits monotonically-sequenced `CosignedRequest` objects; `verify_cosigned_request()` validates signature and payload hash; `detect_omission()` implements §3.3 above. The `Signer` / `SigVerifier` interfaces accept any backend (software Ed25519 for testing, hardware-bound keys in production).

`witness.py` — `Witness.observe(sth)` accepts STHs from the operator, verifies their signatures, and raises `SplitViewError` when two STHs with the same `tree_size` carry different `root_hash` values. `HttpWitnessTransport` submits STHs to independent witness endpoints over stdlib `urllib`; counter-signatures are verified with Ed25519 before counting toward quorum.

Reference implementation and adversarial harness (`docs/demo/`, reproducible, no network / no model). Ed25519 device-bound keys. Nine scenarios over a 5-request stream:

Session	Operator behaviour	Outcome	Ref.
A	Honest — inspects + commits input and output	`detect_omission() == []`	§3
B	Silently skips input inspection of seq=2	`== [2]`, detected by subject alone	§3.3
C	Fakes an input ack (bogus STH + proof)	Rejected; `[2]` surfaces	§3.2
D	Rewrites a past log entry	Consistency proof fails	§3.4
E	Forges a request for a seq never sent	Rejected; signature ≠ registered key	§2.3
F	Signs receipt then omits; request lost in transit	Attributable omission isolated	§6.8
G	Commits input but omits output inspection seq=2	Check 5 fails; cascade	§3.2
H	Shadow routing seq=2 (OSM-042)	Transparent to protocol	§8.2
J	Behavioral drift: rejection heuristics degrade	Canary delta flagged (probabilistic)	§6.11

Table 1: Adversarial harness scenarios. Sessions A–H plus J cover all paper claims.

The harness is anchored in the test suite (`tests/test_completeness_demo.py`, `tests/test_network_reconcil` `tests/test_output_inspection.py`); 1831 tests total, all passing.

5. Related Work

Note on novelty. The subject-monitoring mechanism is NOT novel. We cite its predecessors as our foundation, not as work we improve upon.

CONIKS [9] is the direct ancestor. CONIKS allows users to monitor their own key-binding entries in a transparency log, detecting inconsistency *without global monitors* at low overhead (<20 kB/day). Client monitoring happens automatically in the background, with no explicit user action required per check. Over *key bindings*, CONIKS already delivers nearly everything we propose for inspection logs: subject-side monitoring, automatic silent operation, and reduced dependence on external witnesses.

Key Transparency family. [2, 5, 10, 11] Google Key Transparency (2017), Signal Key Transparency (2023), Apple Contact Key Verification (2023), and WhatsApp Key Transparency (2023) are production deployments of CONIKS-style subject-monitoring. The **IETF `keytrans` Working Group** [4] standardises this family. This is live production work across major platforms; the primitive is mature and being standardised.

Certificate Transparency. [6, 7] RFC 6962 and RFC 9162 provide the append-only log infrastructure and gossip mechanisms for cross-operator STH validation. Our implementation is RFC 9162 faithful. The split-view limit (limit 6) is addressed by RFC 9162 §5 gossip.

The Attacker Moves Second [1] demonstrates formally that adaptive adversaries defeat >90% of existing defences. This supports our decision not to promise detection rates. We claim verifiability of the inspection record, not detection probability.

Our contribution, stated precisely. The CONIKS/KT mechanism monitors *key bindings*; the adversarial event is *equivocation*. We transfer this model to *AI content-inspection logs*; the adversarial event is *omission*. The invariant to protect changes: from "my key binding is consistent across all views" to "every request I signed was preceded by a registered cause for inspection." We add inspection-by-cause (§3.2) as the specific record structure that makes this invariant falsifiable. This is a domain-transfer and modelling contribution, not a cryptographic primitive.

Concurrent Work (January–June 2026)

Notarized Agents [3] (Figuera, 2026) introduces receiver-side signing for multi-agent workflows: each tool or service that receives an agent's call signs a Receipt over what it actually observed, creating a Merkle-anchored audit trail. The threat model is a *compromised agent* fabricating its own traces — not a semi-honest operator omitting inspections. The paper's honest-limits section explicitly acknowledges the set-completeness gap it does not close: "Owner receives verifiable individual receipts but cannot cryptographically prove the log returned *all* matching entries. Operators could silently omit receipts while remaining cryptographically correct on returned ones." This is exactly the gap our mechanism closes. The two contributions are orthogonal in threat model: Sello secures agents against infrastructure they control; we secure subjects against operators who control both the AI pipeline and the inspection log.

Aegon [13] applies CT-style Merkle logs and hardware-attested receipts to *content-licensing provenance* (not AI inspection). Their explicit honest limit: "cannot guarantee completeness against an adversarial SDK operator." Our mechanism closes this via subject-side monotonic sequence monitoring; the completeness property Aegon declares out of scope is the central contribution of this paper.

Auditable Agents [12] (Nian et al., 2026) proposes a Merkle log for agent execution traces, providing post-hoc auditability of tool calls and state transitions. The mechanism assumes *honest tool providers*; completeness against a dishonest operator is not addressed. Our contribution is precisely the case they assume away.

Aegis [8] (Mazzocchetti, 2026) combines ZK-STARK Proof-of-Conduct with an ILK hash chain for governance of autonomous agents. Aegis addresses *behavioral compliance* (did the agent follow rules?), not completeness of an operator-side inspection log. The two mechanisms are complementary.

6. Honest Limits

These limits are not weaknesses to apologise for; they are the boundary conditions that make every other claim credible.

6.1 Split-view: partial mitigation implemented, full closure pending. `detect_omission()` detects gaps in the subject's view of the log. It does not prove that a regulator sees the same view. The HTTP witness transport is implemented (`HttpWitnessTransport`, Ed25519-verified

quorum, `has_quorum()`); deployment of independent witness nodes remains ecosystem work outside this paper's scope.

6.2 Out-of-band content and retained data. The mechanism covers the synchronous request path. Operator-side batch reprocessing of stored prompts, asynchronous safety scans, or any inspection that does not reference a live `CosignedRequest` are not covered.

6.3 Client-operator circularity. The guarantee rests on the independence of the client. If the operator both runs the AI service and distributes the client software, they could ship a client that uses a weak key or a counter that resets silently. The strong guarantee requires the client to be distributed by a party independent of the AI operator.

6.4 Definition gaming. "What constitutes an inspection" is defined by the operator. An operator can comply with the mechanism while defining "inspection" narrowly enough to exclude most meaningful safety checks. The mechanism provides a verifiable record of compliance with the *defined* inspection policy; it does not enforce any particular policy.

6.5 Geolocation and export control are not resolvable in code. IP address, GPS, and round-trip latency are risk signals, not proof of geographic location. What the filter makes verifiable is: "we generated a risk score of X for subject Y on date Z and escalated to KYC." Whether the KYC step resolves residency is the legal system's responsibility.

6.6 Chain-of-thought is not auditable as ground truth. We log `(decision, action, cause)`, not the model's reasoning trace. Large language model chain-of-thought outputs are known to be potentially post-hoc rationalisations. We log the *act* and the *triggering rule*; the act is verifiable and the rule is enumerable. The reasoning is not.

6.7 Retroactive compliance: architectural mitigation, not cryptographic closure. The in-path guarantee is a property of the deployed architecture, not a cryptographic invariant. An operator who deliberately engineers a bypass — letting the model run before the filter commits, then inserting a retroactive record — can defeat it without forging any signature.

6.8 Network failures create plausible deniability for unconfirmed requests. A gap in the subject's sequence has two indistinguishable causes: operator omission, or the request never arrived. The mitigation (OSM-040) is a signed receipt: the operator returns a `Receipt{seq, payload_hash, received_at_ns}` signed on arrival, *before* inspection. A subject who holds a valid receipt for `seq=n` but finds no inclusion proof for `n` has an *attributable omission*. Exercised in Session F and `tests/test_network_reconciliation.py`.

6.9 Legal shield is a hypothesis without legal counsel. Section 7 proposes that verifiable inspection logs create a mutual protection incentive. Whether this argument succeeds in any jurisdiction depends on local law. We present it as a plausible deployment incentive, not a legal fact.

6.10 Covert post-inspection capability restrictions are not detectable. The completeness mechanism ensures every request was inspected; it cannot constrain what the model does *after* inspection returns "allow." On 2026-06-10, Anthropic launched Claude Fable 5 with an undisclosed instruction that silently reduced response quality for certain AI-research queries without informing the user; the instruction was reversed within 24 hours after public disclosure. The episode confirms that a complete and verifiable inspection log — one that recorded "allow"

for each of those requests — would not have surfaced the degradation. Our protocol makes completeness of inspection verifiable; it makes no claim about the faithfulness of the model's subsequent output.

6.11 Behavioral Drift Detection (OSM-054) — Limitations and Open Problems. Three angles are proposed: (A) behavioral delta observation via canary prompts; (B) ex-post consistency proofs; (C) shadow divergence. All three face common barriers: capability masking can be implicit in model weights; false positives are likely when the model improves legitimately; false negatives occur when restrictions operate at sub-threshold probability levels. We state behavioral drift detection as a **future research problem** and an **open investigative tool**, not a closed detection method. See `src/atlas/security/behavioral.py` for the reference implementation.

7. Deployment and Incentives

7.1. Mutual Protection

The verifiable inspection log creates a protection asymmetry that benefits both parties. The operator can produce an inclusion proof when a subject claims harm from AI-generated content: "inspection record for `(seq=n, payload_hash=h)` is in the log at position k, with STH signed at time t." The subject can audit the `cause` field when they claim wrongful blocking. The same log that protects the operator from "you never inspected this" also exposes the operator to "you inspected this without cause." Both parties benefit if and only if the operator is operating honestly within their declared policy.

7.2. EU AI Act Alignment

Article	Mechanism contribution
Art. 9 (risk management)	Cause-based inspection (§3.2) creates auditable per-request risk assessment record.
Art. 12 (record-keeping)	Merkle log + `InspectionRecord` (decision + cause + timestamp); append-only, tamper-resistant, unlimited retention.
Art. 13 (transparency)	Subject-held replica + `detect_omission()` enable individual-request-level transparency without trusting operator.
Art. 14 (human oversight)	Verifiable log makes inspection record independently auditable by regulator or subject.
Art. 26 (deployer obligations)	Read-API (`GET /api/exec/api/v1/log/...`) exposes per-request evidence base for deployer monitoring.
Art. 53 (systemic risk)	Per-request inspection evidence at scale supports documentation obligation.

Table 2: EU AI Act alignment. The filter provides technical evidence; it does not substitute for substantive legal obligations.

7.3. Enforcement Boundary

Breaking the chain — refusing to participate in the protocol — disables the guarantee for that session. The preferred design is *graceful degradation*: chain breaks are logged and flagged for review, not used as hard gates, to avoid converting every network anomaly into a service outage.

8. Conclusion

We have shown that completeness of AI content-inspection logs is *unilaterally falsifiable* by the subject: a gap in the subject's monotonically-signed request sequence is a proof of omission that requires no external auditor and no cooperation from the operator. The mechanism transfers the subject-monitoring model of CONIKS and Key Transparency to a new domain, adds inspection-by-cause as the specific invariant to protect, and extends it symmetrically to output inspection. Concurrent work on agent auditability (Notarized Agents, Auditable Agents, Aegon, Aegis) addresses related but orthogonal problems; none closes the set-completeness gap against a semi-honest operator that this mechanism closes.

Open work. Three threads remain outside this paper's scope. (1) *Independent witness deployment*: the HTTP witness transport is implemented; operating independent witness nodes requires ecosystem coordination beyond a single deployment. (2) *Behavioral faithfulness*: proving that a model's observable behavior on canary inputs predicts its behavior on the full production distribution requires interpretability infrastructure not yet available at API level. (3) *KYC / export-control binding*: the mechanism provides a device-bound signing key; resolving legal residency for export-controlled access requires a KYC step that is operational and legal, not cryptographic.

A. Four-Layer Defense Stack

The completeness mechanism (§§3–6) does not exist in isolation. This appendix synthesises the four-layer stack, states what each layer does not guarantee, and identifies the open research problems no layer currently closes.

The completeness mechanism is one layer of a coordinated defense-in-depth stack.

A.1. Layer 1–2: Completeness Verification (Input + Output)

Enforces symmetric completeness on both the input request and the output response. Omissions are detectable by the subject through `detect_omission()` without relying on the operator. The structural invariants are OSM-007 (input completeness) and OSM-042 (output completeness). Both are closed.

Does not guarantee: fabricated cause fields; split-view attacks.

A.2. Layer 3: Active Defense and Consensus (OSM-031/052/053)

External-witness network following RFC 9162 gossip. A campaign metric (`C_attempts`, `K_attribution`: ≥ 3 consecutive attempts with cosine similarity > 0.7) triggers flagging with co-signed identity trail.

Does not guarantee: per-attempt detection; single-operator witness deployment.

A.3. Layer 4: Behavioral Drift Observation (OSM-054)

A canary framework: fixed probes re-issued periodically; drift in response distribution signals undisclosed model changes. Session J (demo) exercises this. Probabilistic; does not attribute drift to adversarial manipulation vs. legitimate update.

A.4. Synthesis

Open research. Two problems remain outside the scope of this paper and the published literature as of June 2026: (1) *behavioral faithfulness* — proving that a model's behavior on

Layer	Threat closed	Threat not closed
L1–L2 (OSM-007/042)	Inspection omission detectable by subject	Fabricated cause; split-view
L3 (OSM-031/052/053)	Split-view via witnesses; campaign attribution	Per-attempt detection; single-operator witness
L4 (OSM-054)	Behavioral drift early warning (canary, heuristic)	Behavioral faithfulness; covert capability detection

Table 3: Orthogonal defense layers. No layer subsumes another.

a finite canary set predicts its behavior on the full production input distribution; (2) *covert capability detection* — detecting that a model has acquired a capability without access to its weights or training process.

Acknowledgements

Reference implementation: https://github.com/therealronin23/atlas (private; available upon request for reviewer verification). All claims are anchored in 1831 automated tests, mypy `--strict` clean.

References

[1] Anonymous. The attacker moves second: Evaluating the effectiveness of adaptive adversarial attacks. arXiv:2510.09023 [cs.CR], 2024. URL https://arxiv.org/abs/2510.09023. Demonstrates formally that adaptive adversaries defeat >90% of existing defences.

[2] Gary Belvin et al. Google key transparency. Open-source project, Google, 2017. URL https://github.com/google/keytransparency.

[3] Juan Figuera. Notarized agents: Receiver-attested confidential receipts for AI agent actions. arXiv:2606.04193 [cs.CR], June 2026. URL https://arxiv.org/abs/2606.04193. Receiver-side signing for agent tool calls. Threat model: compromised agent. Explicit honest limit: set-completeness against adversarial operator not addressed.

[4] IETF Key Transparency Working Group. Key transparency architecture. IETF Working Group Draft, 2025. Active draft as of 2025–2026. https://datatracker.ietf.org/wg/keytrans/about/.

[5] Apple Inc. imessage contact key verification. Apple Security Research, 2023. URL https://security.apple.com/blog/imessage-contact-key-verification/.

[6] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, IETF, June 2013. URL https://www.rfc-editor.org/rfc/rfc6962.

[7] Ben Laurie, Adam Langley, Emilia Kasper, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, IETF, December 2021. URL https://www.rfc-editor.org/rfc/rfc9162.

[8] Adam Massimo Mazzocchetti. Cryptographic runtime governance for autonomous AI systems: The Aegis architecture for verifiable policy compliance. arXiv:2603.16938 [cs.CR], March 2026. URL https://arxiv.org/abs/2603.16938. ZK-STARK Proof-of-Conduct + ILK hash chain for agent behavioral compliance. Does not address subject-side inspection-log completeness.

[9] Marcela S. Melara, Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. CONIKS: Bringing key transparency to end users. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 383–398, Washington, D.C., 2015. USENIX Association. URL https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/melara.

[10] Signal Messenger. Auditable key directory. Signal blog, 2023. URL https://signal.org/blog/key-transparency/.

[11] Meta. Whatsapp key transparency. Engineering blog, Meta, 2023. URL https://engineering.fb.com/2023/04/13/security/whatsapp-key-transparency/.

[12] Yi Nian, Aojie Yuan, Haiyue Zhang, Jiate Li, and Yue Zhao. Auditable agents. arXiv:2604.05485 [cs.MA], April 2026. URL https://arxiv.org/abs/2604.05485. Merkle log for agent actions. Assumes honest tool providers; completeness against dishonest operator not addressed.

[13] TODO. Todo — arxiv:2604.06693 rate-limited during retrieval; resolve before submission. arXiv:2604.06693, April 2026. URL https://arxiv.org/abs/2604.06693. CT-style Merkle + hardware-attested receipts for content licensing. Explicit honest limit: completeness against adversarial SDK operator not guaranteed.